

Polaris Orders

Schema, Domain, and First Vertical Slice

Ahmed Khan

April 19, 2026

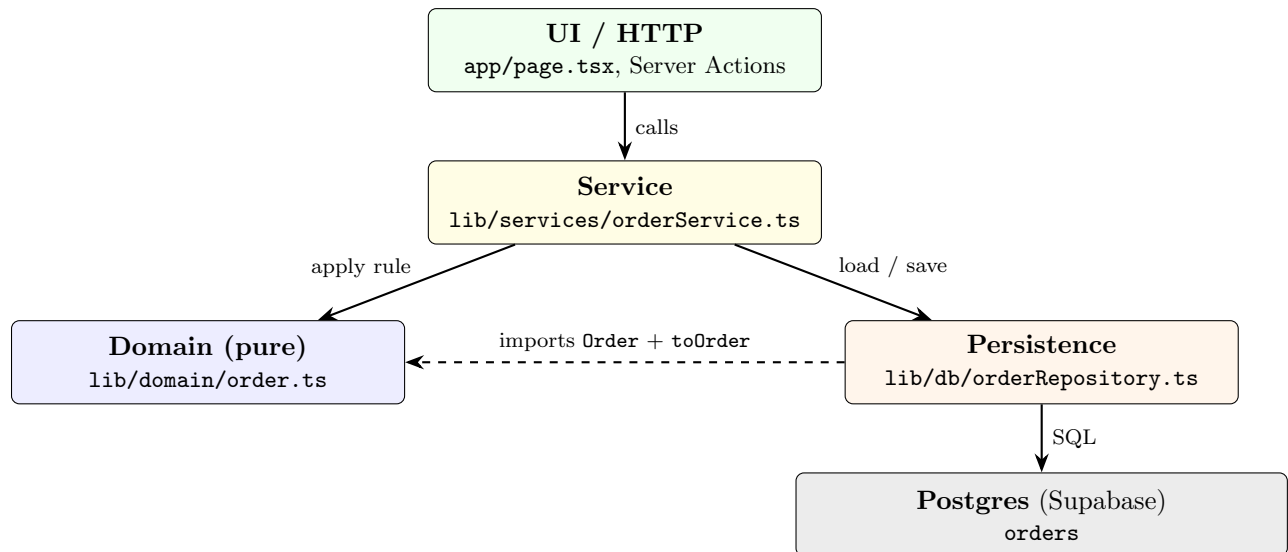
Abstract

This document describes the concrete design of the order subsystem: how the `orders` table is shaped, how the `Order` domain type is derived from it, how the identifier scheme splits internal UUIDs from customer-facing order numbers, and what exists in code today versus what is planned.

General architectural principles behind these choices — the rationale for layered design, why Domain and Persistence are peers rather than a stack, the event-log pattern, and related references — live in `docs/architecture/architecture.md`. Contributor-facing code conventions live in `docs/contributing.md`. Order-number design details live in `docs/order-numbers.md`.

1 Architecture of the Order Slice

The order code is split across four layers, with the Service composing Domain and Persistence from above. The Domain is a leaf: nothing depends downstream of it.



Dependency rule. UI depends on Service; Service depends on Domain and Persistence; Persistence depends on Drizzle, the database, and — narrowly — the Domain (dashed arrow) for the `Order` type and `toOrder` mapper. The Domain depends on nothing. Every arrow points *toward* the Domain. The rationale for this shape and the alternatives considered are in `docs/architecture/architecture.md`.

2 Layer Responsibilities in the Order Slice

Each layer has a contract: what code is allowed to live in it, what is forbidden, and which other layers it may call. Crossings of this contract are review-time red flags.

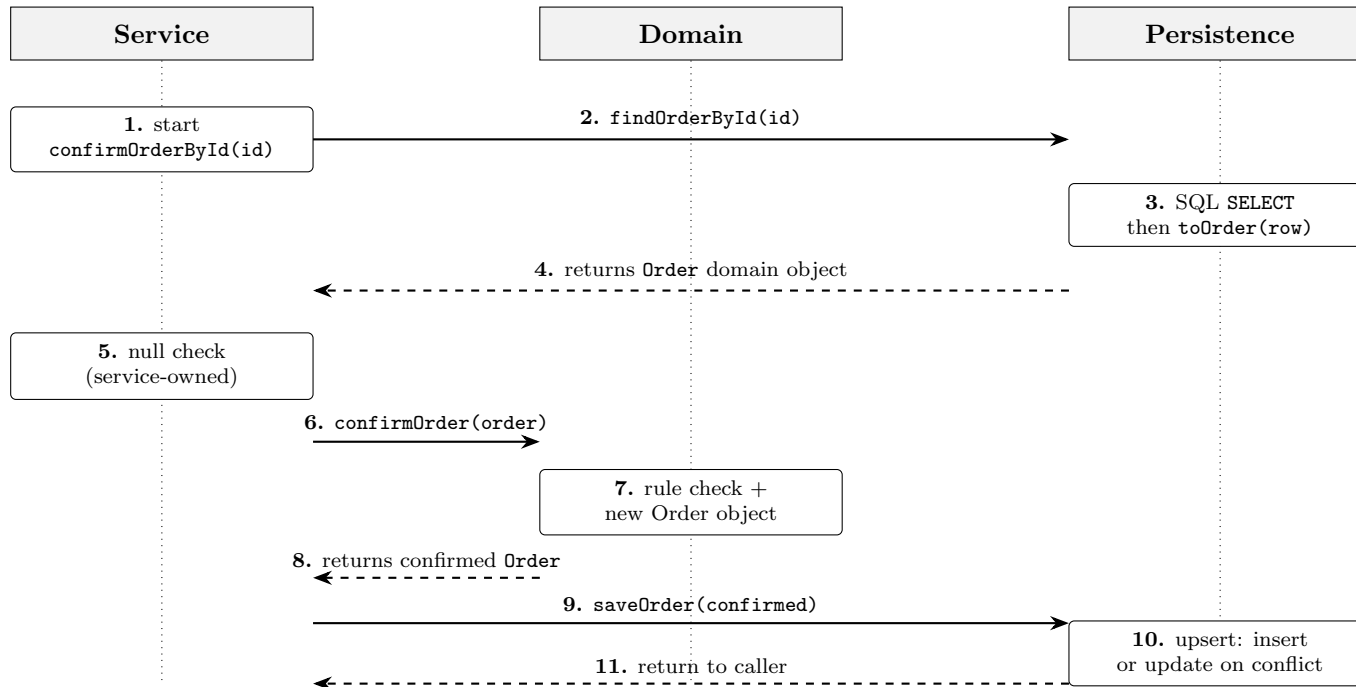
	UI	Service	Domain	Persistence	Database
Lives here	rendering, actions	orchestration, trans- actions	types, pure functions	queries, mapping	tables, constraints, sequences
Forbidden	rules, SQL	SQL	I/O, randomness	rules	—
May call	Service, Persistence	Domain, Persistence	—	Drizzle, Domain	—

Two subtleties worth naming. First, **Persistence depends on Domain**: the repository imports both the `Order` type (as a contract for what it must return) and the `toOrder` mapper (its projection function). Both are pure and depend on nothing below, so the import is narrow and inward-pointing — it does not compromise Domain’s status as a leaf. Second, **reads may skip the Service**: a Server Component rendering a list may call the repository directly, because there is no domain rule to apply and the repository already returns domain-shaped objects. Mutations always go through the Service so any future rule has a natural hook to land in.

3 Mutation Flow for Future Order Operations

Three-beat rhythm: **load** → **apply** → **save**. Rules live in Domain, I/O in Persistence, sequencing in Service.

Status today. The flow below is the pattern order mutations will grow into. The only mutation currently wired (`createOrder`) bypasses load/apply because creation has no precondition to enforce — a UUID is generated and the row is inserted. The load → apply → save shape becomes necessary once status transitions arrive (see [docs/roadmap.md](#)).



Solid arrows are calls; dashed are returns. Steps 3 and 10 touch SQL; steps 6–7 are the only pure-logic beats.

4 First Vertical Slice

4.1 Current State

The Order domain is deliberately minimal — just enough to prove the architecture and exercise the order-number generation scheme. One mutation (`createOrder`), one query (`findAllOrders`), a single pure function (`toOrder` mapping DB rows to the domain shape), and a read-only Server Component rendering a four-column Kanban over seeded orders. Status, transitions, customers, and line items are deferred; see `docs/roadmap.md`.

```
type Order = {
  id: string          // UUID, internal primary key, never rendered to users
  orderNumber: number // bigint, customer-facing, from Postgres sequence
  createdAt: Date     // server-set via DEFAULT now()
}
```

4.2 Identifier Scheme

Every order carries two identifiers with different jobs. `id` is a UUID — a good database key: unguessable, generatable without a round-trip, unaffected by ordering. `orderNumber` is a `bigint` sourced from a Postgres sequence `order_number_seq` starting at 1_000_000 — a good human identifier: short, pronounceable, barcode-friendly, the thing dispatchers read over a radio. Neither compromises for the other's role. The application never generates an order number; the database does, via `DEFAULT nextval('order_number_seq')`. Full rationale and constraints live in `docs/order-numbers.md`.

4.3 File Map

File	Role
<code>lib/domain/order.ts</code>	Order type + <code>toOrder</code> mapper (pure)
<code>lib/db/orderRepository.ts</code>	Drizzle queries; imports <code>toOrder</code>
<code>lib/services/orderService.ts</code>	<code>createOrder</code> (insert only, no domain rule yet)
<code>lib/schema.ts</code>	Drizzle schema: <code>orders</code> + <code>order_number_seq</code>
<code>lib/db.ts</code>	Drizzle client setup
<code>lib/seed.ts</code>	Development seed data
<code>app/page.tsx</code>	Server Component rendering the Kanban
<code>app/actions.ts</code>	<code>createOrderAction</code> Server Action + <code>revalidatePath</code>
<code>drizzle/0001..0003_*.sql</code>	Incremental migrations (<code>posts</code> → <code>orders</code> ; trim columns; add <code>uuid</code> + sequence)
<code>tests/migrations.test.ts</code>	Testcontainers-backed migration smoke test
<code>tests/orderMapper.test.ts</code>	Unit tests for <code>toOrder</code> (incl. leak guard)